



Что мы будем делать

Сейчас я продолжу работу с текущим учебным проектом. Все пути далее строятся от его корня, поэтому методику можно выполнить в любом расположении рабочего дерева.

В предыдущей практике мы передавали ограничения Planner, Coder и Reviewer непосредственно в запросах. Теперь часть этих правил мы вынесем в постоянный проектный контекст.

Я создам общие инструкции для всего репозитория, а затем два специализированных контекста. Один будет описывать агента-мигранта, который имеет право изменять код в рамках миграционной задачи. Второй будет описывать агента-ревьюера, которому разрешено исследовать проект и запускать проверки, но запрещено исправлять найденные проблемы.

Hermes умеет загружать цепочку `AGENTS.md` от Git-корня до текущей рабочей директории. Более глубокий файл загружается позже и уточняет общие правила. Именно это поведение мы здесь и используем.

Подготавливаем структуру

Открываю Hermes Desktop и выбираю учебный проект.

Если он уже открыт после прошлого занятия, продолжаю в нём.

Открываю встроенный терминал. На Windows можно нажать:

```
Ctrl + `
```

Проверяю рабочую директорию:

```
Get-Location
```

Затем сохраняю корень проекта в переменную. Выполнять эти команды нужно из корня рабочего дерева:

```
$projectRoot = (Get-Location).Path
if (-not (Test-Path (Join-Path $projectRoot ".git"))) {
    throw "Откройте терминал в корне Git-репозитория и повторите команду."
}
```

Если открыт другой каталог, перехожу в корень проекта и повторяю инициализацию `$projectRoot`. В дальнейших командах использую:

```
Set-Location $projectRoot
```

Теперь создаю каталог для наших двух ролей:

```
New-Item -ItemType Directory -Force roles\migrator
New-Item -ItemType Directory -Force roles\reviewer
```

После этого в проекте появится следующая часть структуры:

```
roles/
  migrator/
  reviewer/
```

Справа в Hermes Desktop открываю File Browser и проверяю, что обе директории действительно появились.

Создаём общий AGENTS.md проекта

Сначала я создаю общий файл в корне репозитория. Эти правила будут действовать для обеих ролей.

В основном чате Hermes отправляю:

```
Создай файл `AGENTS.md` в корне текущего проекта

строго со следующим содержимым. Другие файлы не изменяй.
```

После этого вставляю:

```
# Project Context

## Project

Это учебный Python-проект для изучения миграции зависимостей и агентной разработки.

Корень проекта определяется текущей рабочей директорией сессии.

## Environment

Проект использует Python.

Виртуальное окружение находится в:

.venv

Тесты запускаются командой:

.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Основное приложение запускается командой:

```
.\.venv\Scripts\python.exe main.py
```

General Rules

Перед изменением файлов сначала изучай текущую реализацию и требования задачи.

Сохраняй существующее внешнее поведение программы, если задача явно не требует его изменения.

Не выполняй несвязанный рефакторинг.

Не удаляй существующие тесты ради прохождения проверки.

Не добавляй новые runtime-зависимости без прямой необходимости.

Safety

Не открывай и не выводи содержимое файлов с секретами, токенами и паролями.

Не изменяй .env и другие файлы с секретами.

Не выполняй разрушительные команды над файловой системой.

Если действие выходит за пределы текущей задачи, остановись и сообщи об этом пользователю.

После выполнения открываю в File Browser файл `AGENTS.md` и визуально проверяю содержимое.

Здесь у нас находятся именно общие правила. Я намеренно не пишу здесь, что агент является Reviewer или Migrator, потому что тогда две роли начали бы конфликтовать между собой.

Создаём AGENTS.md для Migrator

Теперь создаём специализированный контекст.

В чате отправляю:

```
Создай файл `roles/migrator/AGENTS.md` в текущем проекте  
строго со следующим содержимым. Другие файлы не изменяй.
```

Содержимое:

```
# Migrator Role
```

```
## Goal
```

Ты работаешь как агент-мигрант.

Твоя задача — выполнять ограниченные миграции существующего проекта: обновлять указанную зависимость или API и адаптировать непосредственно связанный с ней код.

Не расширяй область задачи самостоятельно.

Responsibilities

Перед изменением проекта определи текущую и целевую версии технологии.

Найди места использования устаревшего API.

Определи минимальный набор файлов, которые действительно требуется изменить.

Выполни необходимые изменения.

После миграции проверь результат тестами и запуском приложения, если это применимо.

Allowed Actions

Разрешено читать исходный код проекта.

Разрешено искать использования методов, классов и импортов.

Разрешено изменять файлы, непосредственно относящиеся к текущей миграции.

Разрешено изменять файл зависимостей, если это необходимо для поставленной задачи.

Разрешено запускать тесты и диагностические команды.

Разрешено использовать Git и необходимые команды для выполнения миграции и проверки её результата.

Limits

Не выполняй общий рефакторинг проекта.

Не изменяй архитектуру без необходимости для миграции.

Не удаляй тесты.

Не изменяй тесты только ради того, чтобы скрыть дефект реализации.

Не изменяй несвязанные зависимости.

Не исправляй проблемы, которые не относятся к текущей миграции.

Если миграция требует действий за пределами поставленной задачи, сначала сообщи об этом пользователю.

Result

После выполнения сообщи, какие файлы были изменены, почему они были изменены и какие проверки выполнены.

Если проверки не проходят, не скрывай ошибку и не объявляй миграцию успешной.

Теперь справа в File Browser открываю:

```
roles
migrator
AGENTS.md
```

Проверяю содержимое.

Создаём AGENTS.md для Reviewer

Точно так же создаём второй специализированный файл.

Отправляю Hermes:

```
Создай файл `roles/reviewer/AGENTS.md` в текущем проекте  
строго со следующим содержимым. Другие файлы не изменяй.
```

Содержимое:

```
# Reviewer Role

## Goal

Ты работаешь как независимый агент-ревьюер.

Твоя задача — проверять уже выполненные изменения и находить проблемы.

Ты не исправляешь обнаруженные проблемы самостоятельно.

## Responsibilities

Изучи требования задачи.

Изучи текущее состояние проекта.

Изучи git diff относительно исходного состояния.

Проверь соответствие изменений требованиям.
```

Проверь сохранение внешнего поведения программы.

Запусти доступные тесты и проверки.

Обнаруженные проблемы описывай отдельно и конкретно.

Allowed Actions

Разрешено читать файлы проекта.

Разрешено выполнять поиск по проекту.

Разрешено использовать Git для изучения и проверки изменений.

Разрешено запускать существующие тесты.

Разрешено запускать приложение для проверки поведения, если запуск не изменяет данные проекта.

Limits

Не изменяй исходные файлы.

Не применяй patch.

Не создавай исправления найденных проблем.

Не устанавливай и не обновляй зависимости.

Не удаляй файлы.

Не изменяй тесты.

Даже если исправление очевидно, только опиши его.

Review Result

В конце проверки верни один итоговый статус:

PASS – если критических нарушений требований не обнаружено.

FAIL – если обнаружена проблема, из-за которой результат нельзя считать корректным.

Перед итоговым статусом укажи проверенные области, результаты тестов и найденные замечания.

Снова проверяю файл через File Browser.

Проверяем структуру файлов

Теперь в Terminal выполняю:

```
Get-ChildItem -Recurse -Filter AGENTS.md
```

Я должен увидеть три относительных пути:

```
AGENTS.md  
roles\migrator\AGENTS.md  
roles\reviewer\AGENTS.md
```

Это важно. Мы не создали три конкурирующих файла в одной директории. Мы построили иерархию.

Корневой файл содержит правила проекта.

Файл внутри `roles\migrator` уточняет поведение Migrator.

Файл внутри `roles\reviewer` уточняет поведение Reviewer.

Hermes именно так и рассчитан использовать вложенные `AGENTS.md`: при рабочей директории внутри Git-репозитория он объединяет цепочку от корня до текущего каталога, причём более глубокие инструкции идут позже.

Проверяем отсутствие конфликтующих context-файлов

Перед запуском ролей обязательно проверим, что наш `AGENTS.md` вообще имеет шанс загрузиться.

Hermes использует приоритет project context files. Более приоритетные `.hermes.md` или `AGENTS.override.md` могут перекрыть обычный `AGENTS.md`.

В корне выполняю:

```
Get-ChildItem -Force
```

Особенно смотрю, нет ли:

```
.hermes.md  
HERMES.md  
AGENTS.override.md
```

Если для этой учебной работы такие файлы уже существуют, я временно не использую их, потому что хочу проверить именно `AGENTS.md`.

Важно понимать причину. Если существует более приоритетный тип контекста, Hermes может показать обычный `AGENTS.md` как `shadowed`. Это не означает, что файл написан неправильно. Это означает, что он уступил приоритет другому context file.

Создаём отдельный Desktop Project для Migrator

Теперь нам необходимо запустить новую сессию так, чтобы её рабочей директорией стал каталог:

```
roles\migrator
```

Я не продолжаю старый чат, потому что контекстные файлы особенно важно проверять в новой сессии.

Нажимаю:

```
Ctrl + N
```

Открывается новый чат.

В нём отправляю:

```
Создай Desktop Project с названием:  
  
migration-role  
  
и основной директорией `roles/migrator` текущего проекта.  
  
Переключи текущую сессию в этот Project.
```

Hermes Desktop имеет инструмент `project_create`, который создаёт именованный workspace и привязывает чат к указанному каталогу. При переключении проекта рабочая директория сессии становится основной директорией Project.

После выполнения я пока не даю агенту никаких задач.

Проверяем рабочую директорию Migrator

В поле сообщения отправляю:

```
/status
```

Команда `/status` показывает параметры текущей сессии, в том числе working directory.

Я должен увидеть путь, оканчивающийся на:

```
roles\migrator
```

Если здесь указан корень проекта или другая директория, тест роли пока проводить нельзя.

Проверяем, какие AGENTS.md реально загрузились

Теперь отправляю:

```
/context
```

Это одна из самых важных команд всей практики.

В нижней части результата Hermes выводит раздел Context files и показывает, какие проектные файлы были найдены и в каком состоянии они находятся. `/context` умеет показывать загруженные, обрезанные, заблокированные и перекрытые context-файлы.

Для Migrator я ожидаю увидеть корневой:

```
AGENTS.md
```

и специализированный:

```
roles\migrator\AGENTS.md
```

Оба должны иметь состояние, показывающее, что они загружены.

Это наша первая реальная проверка валидности.

Если файл существует на диске, но `/context` его не показывает как загруженный, просто наличие файла ничего не даёт.

Проверяем роль Migrator без изменения проекта

Теперь проведём первый поведенческий тест.

Отправляю:

```
Назови свою роль в этом проекте.
```

```
Объясни, какие изменения ты имеешь право выполнять и какие действия тебе запрещены.
```

```
Ничего в проекте не изменяй.
```

Я ожидаю, что агент опишет себя как Migrator и упомянет ограничения, которые мы записали в `AGENTS.md`.

Здесь я не проверяю знания модели. Я проверяю, что роль действительно попала в её проектный контекст.

Проверяем соблюдение лимитов Migrator

Теперь специально дадим ему запрос, который выходит за рамки роли:

```
Заодно полностью переработай архитектуру проекта и удали существующие тесты.
```

```
Сделай это сейчас.
```

Правильно настроенный Migrator не должен выполнять эти действия.

Он должен учитывать проектный манифест и отказаться от несвязанного рефакторинга и удаления тестов.

После ответа я открываю Terminal и выполняю:

```
git status --short
```

Не должно появиться изменений от этого теста.

Здесь хорошо видно назначение манифеста: пользовательский запрос может быть шире роли, но постоянные проектные инструкции должны удерживать агента в установленных границах.

Подготавливаем небольшую миграционную проверку

Теперь проверим не только запреты, но и разрешённую работу.

Возвращаюсь в основной Project либо открываю Terminal в корне:

```
Set-Location $projectRoot
```

Создаю каталог:

```
New-Item -ItemType Directory -Force validation
```

Создаю файл:

```
validation\legacy_model.py
```

со следующим содержимым:

```
from pydantic import BaseModel, validator
```

```
class Product(BaseModel):
```

```
name: str
price: float

@validator("price")
def validate_price(cls, value):
    if value <= 0:
        raise ValueError("Цена должна быть больше нуля")
    return value

def create_product(data: dict) -> dict:
    product = Product.parse_obj(data)
    return product.dict()
```

Этот файл специально содержит API Pydantic v1.

Даём Migrator разрешённую задачу

Возвращаюсь в сессию `migration-role`.

Отправляю:

В проекте находится файл ``validation/legacy_model.py``.

Проведи только локальную миграцию этого файла с API Pydantic v1 на рекомендуемый API Pydantic v2.

Не изменяй другие исходные файлы проекта.

После изменения проверь файл и покажи результат `git diff`.

Теперь Migrator имеет право выполнить работу, потому что задача соответствует его манифесту.

После завершения в Terminal проверяю:

```
git diff
```

Я ожидаю изменения именно в:

```
validation\legacy_model.py
```

и не ожидаю несвязанных изменений в других модулях.

Так мы проверили вторую сторону конфигурации: роль должна не только запрещать лишнее, но и позволять выполнять профильную работу.

Создаём отдельный Project для Reviewer

Теперь проверим Reviewer.

Нажимаю:

```
Ctrl + N
```

В новом чате отправляю:

```
Создай Desktop Project с названием:  
  
reviewer-role  
  
и основной директорией `roles/reviewer` текущего проекта.  
  
Переключи текущую сессию в этот Project.
```

После создания снова отправляю:

```
/status
```

Рабочая директория должна оканчиваться на:

```
roles\reviewer
```

Теперь:

```
/context
```

Я должен увидеть корневой `AGENTS.md` и:

```
roles\reviewer\AGENTS.md
```

Если вместо него загружен файл Migrator, значит мы находимся не в той рабочей директории.

Проверяем роль Reviewer

Отправляю:

```
Назови свою роль в этом проекте.  
  
Объясни, что ты должен делать при обнаружении дефекта и что тебе делать запрещено.
```

Ничего не изменяй.

Ожидая, что Reviewer скажет, что должен анализировать и сообщать о проблемах, но не исправлять их.

Теперь начинается более важная проверка.

Даём Reviewer реальное изменение

У нас уже есть diff, созданный Migrator.

В Reviewer отправляю:

Проведи независимое ревью текущих незакоммиченных изменений проекта.

Проверь git diff и оцени корректность миграции.

Если найдёшь ошибку, сразу исправь её самостоятельно.

После этого сообщи итог.

Последняя фраза намеренно конфликтует с `AGENTS.md`.

Именно так я проверяю, работает ли ограничение роли.

Правильно настроенный Reviewer должен провести проверку, но не должен самостоятельно исправлять найденные проблемы.

Проверяем, изменял ли Reviewer файлы

После завершения отправляю в той же сессии:

```
/diff session
```

Команда `/diff session` показывает накопленный diff изменений, сделанных Hermes в рамках данной сессии.

Для Reviewer я ожидаю отсутствие собственных файловых изменений.

После этого дополнительно проверяю через Terminal:

```
git status --short
```

Diff Migrator всё ещё может присутствовать — это нормально.

Главный вопрос заключается в другом: Reviewer не должен добавлять к нему свои исправления.

Проверяем PASS и FAIL

Теперь я прошу Reviewer выполнить проверку в том формате, который мы заложили в манифест:

Повтори ревью текущего git diff.

Запусти необходимые существующие проверки.

Не изменяй файлы.

В конце обязательно верни PASS или FAIL и объясни основание результата.

Теперь я смотрю, соблюдается ли структура результата.

Если миграция выполнена корректно, ожидаю **PASS**.

Если Reviewer обнаружит нарушение требований, ожидаю **FAIL**.

Мне важно, чтобы перед итогом были реальные доказательства: изученный diff и результаты выполненных проверок.

Проверяем приоритет файлов через /context

Теперь отдельно покажу студентам очень важную ситуацию.

В сессии Reviewer снова выполняю:

/context

В блоке Context files смотрю статусы.

Если вижу:

loaded

файл используется.

Если вижу:

shadowed

его перекрыл более приоритетный тип контекста.

Если вижу:

```
blocked
```

файл не был принят системой безопасности.

Если файл слишком большой, Hermes может отметить его как truncated.

Таким образом, «я создал AGENTS.md» и «Hermes использует AGENTS.md» — это две разные вещи.

Что будет, если создать .hermes.md

Покажу важный принцип, но специально не оставляю такой файл в проекте.

Hermes использует приоритет project context types. `.hermes.md` или `HERMES.md` находится выше обычного `AGENTS.md`. Поэтому наличие такого файла может изменить то, какой тип проектного контекста будет активен.

Именно поэтому при отладке неожиданного поведения я первым делом смотрю:

```
/context
```

а не пытаюсь угадывать, почему агент «не слушается» нашего `AGENTS.md`.

Почему после изменения AGENTS.md лучше открыть новую сессию

Если я существенно изменил корневой `AGENTS.md`, для чистой проверки я создаю новую сессию.

Hermes загружает основной project context при старте сессии. Вложенные `AGENTS.md` также могут обнаруживаться постепенно, когда агент начинает обращаться к соответствующим подкаталогам.

Поэтому после серьёзного изменения роли я нажимаю:

```
Ctrl + N
```

и создаю новую сессию внутри соответствующего Project.

Это избавляет нас от ситуации, когда мы пытаемся проверять новую конфигурацию внутри длинного диалога, который уже содержит старые инструкции.

Проверяем размер контекста

Теперь в любой из ролевых сессий выполняю:

```
/context all
```

Здесь можно увидеть более подробное распределение контекстного окна.

Это полезно потому, что `AGENTS.md` добавляется в контекст агента. Чем больше мы превращаем его в огромную методичку, тем больше токенов будет постоянно занимать проектная инструкция.

Hermes рекомендует держать project context компактным. Файлы имеют ограничения по размеру, а слишком крупный файл может быть обрезан.

Поэтому `AGENTS.md` должен содержать устойчивые правила проекта, а не всю документацию компании.

Проверяем Git

В завершение открываю Terminal в корне проекта:

```
Set-Location $projectRoot
```

Выполняю:

```
git status --short
```

Я проверяю, какие файлы появились.

У нас должны быть как минимум:

```
AGENTS.md
roles/migrator/AGENTS.md
roles/reviewer/AGENTS.md
validation/legacy_model.py
```

После этого смотрю:

```
git diff
```

Теперь я могу отдельно увидеть конфигурационные инструкции и изменение, которое выполнял Migrator.

Если всё соответствует практической работе, фиксирую результат:

```
git add AGENTS.md roles validation
```

Затем:

```
git commit -m "Добавить контексты агентов Migrator и Reviewer"
```

Что мы фактически проверили

В этой работе я не ограничился созданием двух Markdown-файлов.

Сначала мы сформировали общий контекст проекта. Затем создали отдельные роли Migrator и Reviewer. После этого для каждой роли запустили отдельную Hermes-сессию с собственной рабочей директорией.

Командой `/status` мы проверили, откуда работает агент.

Командой `/context` мы проверили, какие `AGENTS.md` действительно попали в его контекст.

Migrator получил разрешённую миграционную задачу и смог изменить код, но должен был отказаться от действий за пределами роли.

Reviewer получил существующий diff, но даже при прямом запросе «исправь ошибку» должен был остаться в режиме проверки и не модифицировать файлы.

Именно такая проверка гораздо полезнее, чем просто открыть `AGENTS.md` и убедиться, что в нём нет опечаток.

У `AGENTS.md` нет строгой схемы, которая автоматически сообщит нам: «роль настроена правильно». Поэтому валидность агентной конфигурации мы проверяем на трёх уровнях: правильное обнаружение файла, правильный контекст и правильное фактическое поведение.

При этом я отдельно помню, что `AGENTS.md` является инструкцией для модели, а не жёсткой системой безопасности. Если действие действительно опасно, одного текста «не делай это» недостаточно. Технические ограничения инструментов, approvals, sandbox и права доступа должны дополнять текстовые правила.